

Contents

CFPB Complaints — Module 3 Lab Guide	2
The Scenario — Your Mission	2
Where you work	2
The problem	3
Your manager’s request	3
Why this is HARD	3
Your team’s job for the next 2 weeks (Module 3)	3
After Module 3	4
How This Guide Works (Read This First!)	4
We do NOT give you the full code	4
Visualizations — Two Modes	4
1. Exploratory charts (for YOU)	4
2. Explanatory charts (for DIANA)	5
Your plotting toolkit (you learned this in Module 2 Class 5)	5
Before You Start — Google Colab Setup	5
Step 1 — Open a new Colab notebook	5
Step 2 — Connect to Google Drive	5
Step 3 — Create a project folder	5
Step 4 — Get the data	6
Step 5 — Test it	6
Colab tips	6
Class 1 — Data Cleaning	7
Your goal	7
Inputs	7
Outputs	7
Phase A — Explore the data first (20 minutes)	7
Phase B — Clean the data (50 minutes)	8
Phase C — Make ONE chart for Diana (15 minutes)	10
Common mistakes in Class 1	10
Self-check before Class 2	11
Class 2 — Encoding and Scaling	11
Your goal	11
Inputs	11
Outputs	11
Phase A — Explore the data first (15 minutes)	11
Phase B — Encode and scale (50 minutes)	12
Phase C — Make ONE chart for Diana (15 minutes)	14
Common mistakes in Class 2	14
Self-check before Class 3	15
Class 3 — Feature Engineering	15
Your goal	15
Inputs	15
Outputs	15
Phase A — Explore the data first (15 minutes)	15
Phase B — Engineer the features (50 minutes)	16
Phase C — Make ONE chart for Diana (15 minutes)	17
Common mistakes in Class 3	17
Self-check before Class 4	18

Class 4 — Feature Selection	18
Your goal	18
Inputs	18
Outputs	18
Phase A — Explore the data first (15 minutes)	18
Phase B — Select features (50 minutes)	19
Phase C — Make ONE chart for Diana (15 minutes)	21
Common mistakes in Class 4	21
Self-check before Class 5	21
Class 5 — Pipelines	21
Your goal	21
Inputs	21
Outputs	22
Phase A — Explore the data first (15 minutes)	22
Phase B — Build the pipeline (50 minutes)	22
Phase C — Make ONE chart for Diana (15 minutes)	24
Common mistakes in Class 5	24
Self-check before Class 6	24
Class 6 — End-to-End Lab (THE BIG ONE)	24
Your goal	24
Time	24
Inputs	25
Outputs	25
Phase A — Explore the data first (15 minutes)	25
Phase B — Build the final dataset (75 minutes)	25
Phase C — Findings report for Diana (10 minutes)	26
Grading rubric (100 points)	27
Submit	27
Tips for the whole module	27

CFPB Complaints — Module 3 Lab Guide

Scenario: US Consumer Financial Protection Bureau. Auto-route citizen complaints to the right department. **Module:** 3 (Data Preparation and Feature Engineering) **Audience:** Adult learners, A2 English. New to AI. **Tools:** Google Colab (no install needed). **Difficulty:** HARD. Multi-class target (10 classes), 4 million rows, very dirty text, big class imbalance, ~5,000 unique companies. Read this guide twice before you start.

The Scenario — Your Mission

Where you work

You and your team are new data analysts at the **US Consumer Financial Protection Bureau (CFPB)**. The CFPB is a US government agency in Washington, DC. The agency protects citizens from unfair banks, credit cards, mortgage lenders, and loan companies.

Every day, citizens write to the CFPB. They are angry. Their bank charged a wrong fee. Their mortgage was rejected without a reason. A debt collector called them 20 times in one day.

Every day: - **10,000 new complaints** arrive at the CFPB. - Each complaint is a long text in English. Citizens type it on the CFPB website. - A human analyst must read each complaint and decide: “Which department should answer this?” - There are **10 departments** (one for credit cards, one for mortgages, one for student loans, and so on). - A human takes **5 minutes per complaint**. That is **833 hours of human work per day**.

The problem

The CFPB is **6 months behind**. Two million complaints sit in a queue, unread.

When the CFPB is slow: - Citizens get angry. They write to their Congressperson. - Banks are not punished for bad behavior. - The press writes negative articles about the agency. - The agency budget gets cut next year.

The Director is in trouble. She walks into your team's office on Monday morning.

Your manager's request

Your manager, **Diana** (Director of Consumer Response at CFPB), tells you:

"Citizens send us complaints about banks, credit cards, mortgages, loans. Ten thousand arrive every day. My team reads each one and sends it to the right department.

A human takes 5 minutes per complaint. We are six months behind.

Build me a model. The model reads the complaint narrative. The model predicts the right product category. Ten categories. Mortgage, credit card, student loan, debt collection, and so on.

If the model is correct **80% of the time**, my team only has to check the 20% the model is unsure about. That is 5x faster. We catch up in six weeks, not six months."

Why this is HARD

This scenario is **harder** than the other labs. Be ready.

Problem	Why it is hard
10 target classes , not 2	A "yes/no" model is easy. A "pick 1 of 10" model is much harder.
65% of narratives are EMPTY	Citizens often check a box but write nothing. We must still use those rows for Modules 3 and 4.
Mortgage = 30% of complaints	The biggest class is much bigger than the smallest. The model will be lazy and predict "Mortgage" for everything.
5,000 unique companies	One-hot encoding makes 5,000 new columns. Table explodes. We must use target encoding .
4 million rows Text has "XXXX"	Too big for free Colab memory. We sample 200,000. CFPB redacts (hides) names, account numbers, addresses. The model sees "XXXX" instead of "John Smith".
Two date columns	date_received and date_sent_to_company. Both stored as text. Both need parsing.
timely_response is 97% "Yes"	Severe imbalance. Almost useless as a target, but a fine feature.

You will see these problems again and again. Do not panic. Solve them one by one.

Your team's job for the next 2 weeks (Module 3)

Diana cannot do this alone. Her data is **one huge CSV file** with 4 million rows. The text is messy. Many columns are missing. The target column has 10 values, not 2.

Your job in Module 3: > **Turn one giant messy CSV into ONE clean parquet file with ~200,000 rows and 20-22 specific columns. The clean file will be used to train the model in Module 4.**

The clean file is called `cfpb_complaints_clean.parquet`. The exact column list is in Class 6.

After Module 3

Module	What happens
Module 4	Train the 10-class classifier on the metadata columns. Diana finally gets her “auto-route” model.
Module 5	Find groups of complaints (debt collection storms, mortgage waves). For management dashboards.
Module 7	Read the actual complaint narrative text. Use NLP. This is where the model becomes really powerful.

You use the **same CFPB dataset** until the end of Module 7.

How This Guide Works (Read This First!)

We do NOT give you the full code

In this guide you will see two kinds of content:

1. WHAT / WHY / EXPECTED — explained in words

For every step you read: - **WHAT** you have to do (in normal language) - **WHY** you do it (the reason) - **HINTS** — the function names, the arguments to remember - **EXPECTED** — the result you should see if you did it right

2. Sometimes a tiny code skeleton with blanks

For the harder steps we give you a code skeleton with ____ blanks. **You fill in the blanks.**

Why no full code?

Because you will NOT learn if you copy-paste. - In Module 4, the model only works if YOU understand each step. - In your future job, nobody will give you the code. - Writing the code yourself takes 10 minutes more today, but saves 10 hours later.

Rule: If you finish a step in 30 seconds by copy-pasting from a teammate’s notebook, do it AGAIN by yourself. Different variable names, same logic.

Visualizations — Two Modes

In every class you will make charts. There are **two reasons** to make a chart:

1. Exploratory charts (for YOU)

Made BEFORE you clean the data. To understand what the data looks like. - Fast, ugly, no labels needed. - Examples: `df.hist()`, `df['column'].value_counts().plot.bar()`.

Goal: “**What does this data look like?**”

2. Explanatory charts (for DIANA)

Made AFTER you understand. To EXPLAIN something to your manager. - Clean, labeled, one clear message. - Must have: title, x-label, y-label, source, and a 1-sentence takeaway.

Goal: “**Diana, look at this. This is the problem.**”

In every class you make BOTH kinds.

Your plotting toolkit (you learned this in Module 2 Class 5)

Plot	When to use it	Function name
Histogram	See the SHAPE of a numeric column	<code>plt.hist()</code> or <code>sns.histplot()</code>
Bar chart	Count of each category	<code>df['col'].value_counts().plot.bar()</code>
Box plot	See outliers in a numeric column	<code>sns.boxplot()</code>
Scatter plot	See the relationship between 2 numbers	<code>plt.scatter()</code> or <code>sns.scatterplot()</code>
Heatmap	See correlation between many columns	<code>sns.heatmap(df.corr())</code>
Line plot	See change over time	<code>plt.plot()</code>
Stacked bar	Show product mix across states	<code>df.groupby(...).unstack().plot.bar(stacked=True)</code>

Before you start: `import matplotlib.pyplot as plt` and `import seaborn as sns`.

Before You Start — Google Colab Setup

You will work in **Google Colab**. That means: - You do NOT install Python. - You do NOT install pandas, scikit-learn, or anything. - You just open a notebook in your web browser.

Step 1 — Open a new Colab notebook

1. Go to <https://colab.research.google.com>
2. Sign in with your Google account.
3. Click “**New notebook**” (top left).
4. Name it `cfpb_module_3.ipynb`.

Step 2 — Connect to Google Drive

Colab files **disappear** when you close the browser. So you must save your work to **Google Drive**.

In the first cell:

```
from google.colab import drive
drive.mount('/content/drive')
```

A pop-up asks for permission. Click “Allow”. After this, Drive is available at `/content/drive/MyDrive/`.

Step 3 — Create a project folder

In a new cell:

```
import os
os.makedirs('/content/drive/MyDrive/cfpb_lab', exist_ok=True)
%cd /content/drive/MyDrive/cfpb_lab
```

Your team will save ALL files in this folder.

Step 4 — Get the data

The CFPB publishes the data on its official portal. It is **US Public Domain**. Free for anyone.

Source URL: <https://www.consumerfinance.gov/data-research/consumer-complaints/>

Option A — Direct download in Colab (recommended):

The full file is about 1.5 GB. Too big for free Colab memory. So we download and **sample 200,000 rows immediately**.

```
import pandas as pd

URL = "https://files.consumerfinance.gov/ccdb/complaints.csv.zip"
# Read directly. Use low_memory=False for mixed-type columns.
raw = pd.read_csv(URL, low_memory=False)
print(raw.shape)          # Should be ~ (4,000,000+, 18)

# Sample 200,000 rows. Fix the random_state so the sample is the same every run.
df = raw.sample(n=200_000, random_state=42).reset_index(drop=True)
print(df.shape)           # (200000, 18)

# Save the sample to Drive so we never download again.
df.to_csv('/content/drive/MyDrive/cfpb_lab/complaints_sample_200k.csv', index=False)
```

Option B — Download by hand:

1. Go to <https://www.consumerfinance.gov/data-research/consumer-complaints/>
2. Click “Download CSV”. Wait for the zip file.
3. Unzip on your laptop.
4. In Colab’s file panel (left sidebar), drag the CSV in.
5. Run the same `.sample()` code as above.

Step 5 — Test it

```
import pandas as pd
df = pd.read_csv('/content/drive/MyDrive/cfpb_lab/complaints_sample_200k.csv',
                 low_memory=False)
print(df.shape)
print(df.columns.tolist())
print(df['Product'].value_counts().head(10))
```

You should see ~200,000 rows. The column Product should have ~10 categories. The biggest is “Mortgage” or “Debt collection” or “Credit reporting”.

Colab tips

Tip	Why
Save your notebook to Drive often Save intermediate .parquet files to Drive Sample EARLY and save	Colab disconnects after 12 hours. Files in /content/ disappear when Colab disconnects. The full 4M-row CSV will crash free Colab. Sample to 200k once, save, reuse.
Use <code>low_memory=False</code> on <code>read_csv</code>	Some CFPB columns mix text and numbers. Without this flag, pandas guesses wrong.
Restart runtime if memory full	Runtime → Restart runtime.

Class 1 — Data Cleaning

Scenario reminder: Diana drops a 1.5 GB CSV file on your desk. There are 18 columns. Two date columns are stored as text. 65% of complaint narratives are empty. Some companies appear once, some appear 50,000 times. Your job today: shrink, clean, understand.

Your goal

Make the messy CFPB file USABLE. Fix the date columns. Find missing values. Understand the 10-class target.

Inputs

- `complaints_sample_200k.csv` in your Drive folder (saved in Setup Step 4)

Outputs

- `complaints_step1.parquet` saved in your Drive folder
 - 3+ exploratory charts in your notebook
 - 1 explanatory chart for Diana
 - Notes in markdown about every decision you made
-

Phase A — Explore the data first (20 minutes)

Before you clean anything, **LOOK** at the data.

Exploratory chart 1 — Count of each Product (the target)

- **Question:** “Of 200,000 complaints, how many are about each product?”
- **HINTS:**
 - Use `df['Product'].value_counts()`.
 - Then `.plot.bar()` on the result.
 - Rotate x-labels with `plt.xticks(rotation=45, ha='right')`.
- **What you learn:** “Mortgage” and “Credit reporting” each take ~30% of the data. The smallest class might be ~1%. This is a **big imbalance**.

Exploratory chart 2 — How many narratives are empty?

- **Question:** “What fraction of the Consumer complaint narrative column is blank?”
- **HINTS:**
 - Use `df['Consumer complaint narrative'].isna().mean()`.
 - This gives you a fraction between 0 and 1.
 - Plot as a 2-bar chart: “Has narrative” vs “Empty”.
- **What you learn:** About **65% are empty**. That is a lot. We must NOT drop them: they still have product, date, company, state. We use them in M3 and M4, then filter to non-empty for M7.

Exploratory chart 3 — Complaints over time (by month)

- **Question:** “When did people complain the most? Was there a spike around the 2008 financial crisis? Around COVID-19 in 2020?”
- **HINTS:**
 - First convert `Date received` to `datetime` (you do this in Phase B Step 3 anyway, do it here too with a `temp` variable).
 - Use `.dt.to_period('M')` to group by month.
 - Plot the counts as a line plot.

- **What you learn:** Big spikes show real-world events. The model should be aware that “year” is a feature.

Exploratory chart 4 — Missing values per column

- **Question:** “Which of the 18 columns has the most missing data?”
 - **HINTS:**
 - Use `df.isna().sum().sort_values(ascending=False)`.
 - Plot as a bar chart.
 - **What you learn:** Consumer complaint narrative, Consumer disputed?, Sub-product will be near the top.
-

Phase B — Clean the data (50 minutes)

Step 1 — Load the sample

- **WHAT:** Load `complaints_sample_200k.csv` into a DataFrame named `df`.
- **HINTS:**
 - Use `pd.read_csv('/content/drive/MyDrive/cfpb_lab/complaints_sample_200k.csv', low_memory=False)`.
- **EXPECTED:** `df.shape` is `(200000, 18)`.

Step 2 — Rename columns

The CFPB column names are LONG and have spaces (e.g., Consumer complaint narrative). Rename them to short snake_case names.

- **WHAT:** Use `df.rename(columns={...})` or `df.columns = [...]`.
- **HINTS:** A dict is cleanest. Skeleton:

```
rename_map = {
    'Date received': 'date_received',
    'Product': 'product',
    'Sub-product': 'sub_product',
    'Issue': 'issue',
    'Sub-issue': 'sub_issue',
    'Consumer complaint narrative': 'narrative',
    'Company public response': 'company_public_response',
    'Company': 'company',
    'State': 'state',
    'ZIP code': '___', # YOU fill in
    'Tags': '___',
    'Consumer consent provided?': '___',
    'Submitted via': 'submitted_via',
    'Date sent to company': 'date_sent_to_company',
    'Company response to consumer': 'company_response_type',
    'Timely response?': 'timely_response',
    'Consumer disputed?': 'consumer_disputed',
    'Complaint ID': 'complaint_id',
}
df = df.rename(columns=rename_map)
```

- **EXPECTED:** `df.columns.tolist()` shows all short names.

Step 3 — Fix the two date columns

- **WHAT:** date_received and date_sent_to_company are stored as **text** (e.g., "2021-08-15"). Convert both to real datetime.
- **HINTS:**
 - The function is `pd.to_datetime()`.
 - Add the argument `errors='coerce'`. If a cell is bad, it becomes NaT (Not a Time = missing). The code does not crash.
 - Use a for loop over the 2 column names. Do NOT write 2 separate lines.
- **WHY:** If dates are strings, you cannot subtract them. "How many days from date_received to date_sent_to_company?" is impossible.
- **EXPECTED:** `df.dtypes` shows `datetime64[ns]` for those 2 columns.

Step 4 — Find missing values

- **WHAT:** Count missing values per column.
- **HINTS:** `.isna()` returns True/False per cell. `.sum()` counts the Trues per column.
- **EXPECTED:** Something like:

```
narrative           ~130,000
consumer_disputed   ~80,000
sub_issue           ~70,000
...
```

Step 5 — Decide what to do with each missing column

You do NOT drop rows with missing values. You handle each column.

Column	Decision	Why
narrative	KEEP missing as empty string "".	We still need the row for product prediction.
consumer_disputed	Make a new column <code>has_narrative_flag = 1</code> if not empty, else 0. Map "Yes"=1, "No"=0, missing=-1 (a 3rd category).	Missing means "field was removed by CFPB after 2017". That itself is information.
sub_issue	Fill with "unknown".	Just a label.
state	Fill with "XX".	Some complaints have no state.
tags	Drop the column. Too many missing values, low information.	We will not use it.
zip_code	Drop the column. Privacy-redacted in most rows.	Not useful.

- **HINTS:**
 - `df['narrative'] = df['narrative'].fillna('')`.
 - `df['has_narrative_flag'] = (df['narrative'] != '').astype(int)`.
 - For `consumer_disputed`: `df['consumer_disputed'] = df['consumer_disputed'].map({'Yes': 1, 'No': 0}).fillna(-1).astype(int)`.

Step 6 — Standardize timely_response

- **WHAT:** This column is "Yes" / "No". Convert to 1 / 0.
- **HINTS:** `df['timely_response'] = (df['timely_response'] == 'Yes').astype(int)`.
- **WARNING:** About **97% of values are "Yes"**. This is extreme imbalance. The column is a poor target but is OK as a feature. Mention this in your markdown notes.

Step 7 — Standardize product values

- **WHAT:** Some product names have been renamed by CFPB over the years. For example, old data has "Credit reporting" but newer data has "Credit reporting, credit repair services, or other personal consumer reports". Merge them.
- **HINTS:**
 - Print all unique values: `df['product'].unique()`.
 - Write a small dict to map old → new.
 - Apply with `.replace(product_map)`.
- **EXPECTED:** After cleaning, you have **exactly 10 product categories**. Write the final list in your notebook.

Step 8 — Write down what you did

In a markdown cell, write: - Starting rows: 200,000 - Rows after cleaning: 200,000 (we kept all) - Rows dropped: 0 - Columns dropped: tags, zip_code - New columns created: has_narrative_flag - WHY for each decision.

Step 9 — Save to Drive

- **WHAT:** Save the cleaned df as parquet, inside your Drive folder.
- **HINTS:** `df.to_parquet('/content/drive/MyDrive/cfpb_lab/complaints_step1.parquet')`.
- **WHY parquet, not CSV?** Parquet is 10x smaller and keeps the dtype info. CSV does not.

Phase C — Make ONE chart for Diana (15 minutes)

She does not have time to read your code. She wants ONE picture.

Diana's chart — "Complaint volume over time"

A line chart showing how the number of complaints has changed every month since CFPB opened in 2011.

- **HINTS:**
 - Group by month: `df.groupby(df['date_received'].dt.to_period('M')).size()`.
 - `.plot.line()`.
 - Add the title: "Monthly CFPB complaint volume – sample of 200k".
 - X-label: "Month".
 - Y-label: "Number of complaints".
- **Takeaway for Diana:** "Complaint volume has tripled since 2015. The 2020 COVID spike is huge. Your backlog is not random; it follows real economic events."

Common mistakes in Class 1

Mistake	What goes wrong
Forget <code>errors='coerce'</code> on <code>to_datetime</code>	Code crashes on one bad date.
Drop rows where narrative is empty	You lose 65% of the data. Diana fires you.
One-hot encode company here	5,000 columns. Memory crash. We use target encoding in Class 2.
Forget <code>low_memory=False</code> on <code>read_csv</code>	Some columns get dtype object mixed with NaN. Filters break.
Save to <code>/content/</code> instead of Drive	File disappears when Colab disconnects.

Self-check before Class 2

- ☐ `complaints_step1.parquet` exists in Drive.
 - ☐ Columns are renamed to `snake_case`.
 - ☐ `date_received` and `date_sent_to_company` have dtype `datetime64`.
 - ☐ `has_narrative_flag` exists. About 35% of rows = 1.
 - ☐ `product` has exactly 10 unique values.
 - ☐ At least 3 exploratory charts in your notebook.
 - ☐ 1 polished chart for Diana.
 - ☐ You wrote down (in markdown) what you did and WHY.
-

Class 2 — Encoding and Scaling

Scenario reminder: Diana looks at your cleaned data. She is happy. But she says: “The model is a math model. It does not understand the word ‘Wells Fargo’ or the word ‘Mortgage’. Turn the words into numbers. But careful — company has 5,000 unique values. Do not blow up the table.”

Your goal

Turn TEXT columns into numbers. Pick the right encoding for each. **Target-encode** the company column. Make narrative-length-style features.

Inputs

- `complaints_step1.parquet` from Class 1

Outputs

- `complaints_step2.parquet` in Drive
 - 3+ exploratory charts
 - 1 explanatory chart for Diana
-

Phase A — Explore the data first (15 minutes)

Exploratory chart 1 — Top 20 companies by complaint count

- **Question:** “Of 5,000 companies, who gets the most complaints?”
- **HINTS:**
 - `df['company'].value_counts().head(20).plot.barh()`.
- **What you learn:** A few mega-banks (Equifax, Bank of America, Wells Fargo) dominate. Most companies have only 1 or 2 complaints. This is a **long tail** distribution. One-hot will not work.

Exploratory chart 2 — Distribution of complaints per company (log scale)

- **Question:** “How long is the tail?”
- **HINTS:**
 - `counts = df['company'].value_counts()`.
 - `plt.hist(counts, bins=50)`.
 - Use `plt.yscale('log')` to see both small and big bars.
- **What you learn:** Hundreds of companies have only 1 complaint. Hundreds have 10. A few have 50,000. We need a smart encoding.

Exploratory chart 3 — submitted_via counts

- **Question:** “Do people complain on the web, by phone, by mail?”
- **HINTS:** `value_counts().plot.bar()`.
- **What you learn:** ~80% are web. Phone is small. This column has low information but does not hurt.

Exploratory chart 4 — company_response_type counts

- **Question:** “How do companies usually respond?”
 - **HINTS:** `value_counts().plot.bar()`.
 - **What you learn:** “Closed with explanation” is most common. “Untimely response” is small. Useful for one-hot.
-

Phase B — Encode and scale (50 minutes)

Step 1 — Split into train and test FIRST

- **WHAT:** Use `train_test_split` BEFORE any encoding. Otherwise you leak test data.
- **HINTS:**
 - `from sklearn.model_selection import train_test_split.`
 - `X = df.drop('product', axis=1).`
 - `y = df['product'].`
 - `train_test_split(X, y, test_size=0.2, random_state=42, stratify=y).`
- **WHY stratify=y?** With 10 classes, the test set may by accident miss small classes. Stratify keeps the same class proportions in train and test.
- **EXPECTED:** `X_train.shape` is (160000, ~18). `X_test.shape` is (40000, ~18).

Step 2 — Label-encode the target product

- **WHAT:** The target has 10 string values. Convert to integers 0..9.
- **HINTS:**
 - `from sklearn.preprocessing import LabelEncoder.`
 - `le = LabelEncoder().`
 - `y_train_enc = le.fit_transform(y_train).`
 - `y_test_enc = le.transform(y_test).`
 - SAVE the encoder. You need it later to decode predictions back to product names.
- **EXPECTED:** `y_train_enc` is an array of integers 0..9. `le.classes_` is the list of 10 product names.

Step 3 — One-hot encode the SMALL categorical columns

- **WHAT:** For columns with **fewer than 20 unique values**, one-hot encoding is fine.
- **The list:**
 - `submitted_via` (~5 values)
 - `company_response_type` (~8 values)
 - `state` (~52 values, including territories — borderline, but still OK for one-hot)
- **HINTS:**
 - `pd.get_dummies(X_train, columns=['submitted_via', 'company_response_type'], prefix=['via', 'resp']).`
 - Do the SAME thing to `X_test` (or use `OneHotEncoder` to be safe — see Class 5).
- **WARNING:** If you use `pd.get_dummies` separately on train and test, you may get DIFFERENT columns. To be safe, use `pd.get_dummies` on the concatenated data, then split back. Or use `sklearn.OneHotEncoder` with `handle_unknown='ignore'` (the safe way).

Step 4 — Target-encode company (the BIG categorical)

This is the most important step in Class 2. **Read it twice.**

- **WHAT:** Replace each company name with a number: the AVERAGE class probability for that company. Since the target has 10 classes, the simplest target encoding is the **most common product** per company, mapped to that product's integer.
- **WHY target encoding?** 5,000 companies means 5,000 new one-hot columns. The table would have 5,000 + 18 = 5,018 columns. Most are zero. Slow and useless. Target encoding gives **ONE** new column instead.
- **HINTS:**
 - For each company in X_train, compute the mean of y_train_enc.


```
# Compute on TRAIN only.
company_target_map = (
    pd.DataFrame({'company': X_train['___'],
                  'y': y_train_enc})
    .groupby('___')['y']
    .mean()
)
# Apply to train AND test.
X_train['company_target_encoded'] = X_train['company'].map(company_target_map)
X_test['company_target_encoded'] = X_test['company'].map(company_target_map)
# Test may have NEW companies not seen in train. Fill them with the overall mean.
overall_mean = y_train_enc.mean()
X_test['company_target_encoded'] = X_test['company_target_encoded'].fillna(____)
```
 - Then DROP the original company column.
- **WARNING — leakage:** You **MUST** compute the mean on TRAIN ONLY. If you compute on all data, the test rows see information from themselves. The model looks great here and fails in production.
- **EXPECTED:** X_train['company_target_encoded'] is a float column with no missing values.

Step 5 — Engineer narrative-related numeric features

The narrative text is for Module 7. But we can extract simple NUMBERS from it now — these help Module 4 even without NLP.

New column	What it measures
narrative_length	Number of characters. 0 if empty.
narrative_n_caps_words	Number of ALL-CAPS words (signals an angry complaint).
narrative_intensity_score	A combined score: caps words divided by total words.
n_dollar_mentions	How many \$ signs in the narrative.
n_dates_mentioned	How many dates appear (regex like <code>\d{1,2}/\d{1,2}/\d{2,4}</code>).

- **HINTS:**
 - `df['narrative_length'] = df['narrative'].str.len().fillna(0).astype(int)`.
 - For caps words: split on space, count words where `w.isupper()` and `len(w) >= 2`.
 - For dollars: `df['narrative'].str.count(r'\$')`.
 - For dates: `df['narrative'].str.count(r'\d{1,2}/\d{1,2}/\d{2,4}')`.
 - For intensity: `n_caps_words / (n_total_words + 1)`. The +1 avoids divide-by-zero.
- **WARNING:** Narratives have many XXXX tokens (CFPB redacted private info). XXXX is ALL CAPS. Do not count XXXX as an angry caps-word. Filter them out first: `narrative.replace('XXXX', ' ')` before counting caps words.

Step 6 — Engineer date-difference feature

- **WHAT:** Compute `days_to_company_response = date_sent_to_company minus date_received`.
- **HINTS:**
 - Subtract two datetime columns. Result is a Timedelta.
 - Use `.dt.days` on the Timedelta to get a number.
 - Some values are negative or huge (data error). Clip them: `.clip(lower=0, upper=365)`.

- **WHY:** If CFPB took 30 days to send the complaint, the citizen is angry by now. Useful for the model.

Step 7 — Log-transform narrative_length

- **WHAT:** narrative_length is very skewed (many zeros, a few huge values). Apply `np.log1p()`.
- **HINTS:** `df['log_narrative_length'] = np.log1p(df['narrative_length'])`.
- **WHY log1p and not log?** `log(0)` is `-inf`. `log1p(x) = log(x + 1)` is safe for zeros.

Step 8 — Scale numeric columns (preview only)

- **WHAT:** Use `StandardScaler` to make every numeric column have mean = 0, std = 1.
- **HINTS:**
 - `from sklearn.preprocessing import StandardScaler.`
 - `scaler = StandardScaler().`
 - `scaler.fit_transform(X_train[numeric_cols]).`
- **WARNING:** This is for inspection only. In Class 5, scaling goes INSIDE the Pipeline. Do NOT save the scaled version as your final file.

Step 9 — Save

- **WHAT:** Save to Drive. `df.to_parquet('/content/drive/MyDrive/cfpb_lab/complaints_step2.parquet')`.
- **Save the UNSCALED version.** Scaling will happen inside the Pipeline later.

Phase C — Make ONE chart for Diana (15 minutes)

Diana's chart — "Top 10 most complained-about companies"

A horizontal bar chart showing the 10 companies that get the most complaints. These are the companies CFPB is watching closest.

- **HINTS:**
 - `df['company'].value_counts().head(10).plot.barh().`
- **Title:** "Top 10 companies by complaint volume – sample of 200k".
- **X-label:** "Number of complaints".
- **Y-label:** "Company name".
- **Takeaway for Diana:** "Three credit bureaus (Equifax, Experian, TransUnion) get 40% of all complaints. Your team should pre-build templates for these three."

Common mistakes in Class 2

Mistake	What goes wrong
Target-encode BEFORE the train/test split	Leakage. Encoder sees test labels. Model looks great, fails in production.
One-hot encode company	5,000 new columns. Memory crash.
Scale BEFORE train/test split	Leakage. Scaler sees test data.
Use <code>np.log</code> instead of <code>np.log1p</code>	<code>np.log(0) = -inf</code> . Crash on empty narratives.
Count XXXX as caps words	Every narrative looks angry. Useless feature.
Save the scaled version	Class 5 will scale again inside the pipeline. Double scaling = wrong numbers.

Self-check before Class 3

- ☐ Train/test split done FIRST.
 - ☐ product is label-encoded (0..9).
 - ☐ company_target_encoded is a numeric column. Original company dropped.
 - ☐ Small categoricals one-hot encoded.
 - ☐ narrative_length, narrative_n_caps_words, narrative_intensity_score, n_dollar_mentions, n_dates_mentioned exist.
 - ☐ days_to_company_response exists.
 - ☐ At least 3 exploratory charts.
 - ☐ 1 chart for Diana.
 - ☐ complaints_step2.parquet saved to Drive.
-

Class 3 — Feature Engineering

Scenario reminder: Diana says: “The raw columns are not enough. The TRULY useful columns are not there. We must MAKE them. For example: which year the complaint was sent, which month, how long the narrative is. Many people forget that **date** is a goldmine.”

Your goal

Make NEW columns from the existing ones. These will help the model predict the product category.

Inputs

- complaints_step2.parquet

Outputs

- complaints_step3.parquet in Drive
 - 3+ exploratory charts
 - 1 explanatory chart for Diana
-

Phase A — Explore the data first (15 minutes)

Exploratory chart 1 — narrative_length by product

- **Question:** “Are mortgage complaints longer than credit-card complaints?”
- **HINTS:**
 - `sns.boxplot(data=df, x='product', y='narrative_length')`.
 - Rotate x-labels: `plt.xticks(rotation=45, ha='right')`.
 - Limit y-axis: `plt.ylim(0, 5000)` (hides extreme outliers).
- **What you learn:** Mortgage narratives are usually longer (people explain a 30-year loan in detail). Debt collection narratives are short and angry.

Exploratory chart 2 — Complaint counts per year

- After you make date_received_year (Step 2 below), plot.
- **Question:** “Which years have most complaints? Did 2020 (COVID) spike?”
- **HINTS:**
 - `df['date_received_year'].value_counts().sort_index().plot.bar()`.
- **What you learn:** Big spikes show real-world events. The model should use year as a feature.

Exploratory chart 3 — Product mix per month of year

- **Question:** “Are mortgage complaints more common in January?”
- **HINTS:**
 - After Step 2, `pd.crosstab(df['date_received_month'], df['product'])`.
 - Plot as a stacked bar.
- **What you learn:** Some products are seasonal. Student loans spike in August (school starts).

Exploratory chart 4 — n_dollar_mentions distribution by product

- **Question:** “Which products mention \$ the most?”
- **HINTS:** Boxplot, `x=product`, `y=n_dollar_mentions`.
- **What you learn:** Mortgage and debt collection mention dollars often. Credit-reporting complaints mention dollars rarely. This is a **strong signal** for the model.

Phase B — Engineer the features (50 minutes)

Step 1 — Re-load the Class 2 file

- **WHAT:** Load `complaints_step2.parquet` into `df`.
- **HINTS:** `pd.read_parquet(...)`.

Step 2 — Date-derived features from `date_received`

Make these new columns:

New column	What it is
<code>date_received_year</code>	The year (2011..present)
<code>date_received_month</code>	The month (1..12)
<code>date_received_dayofweek</code>	0=Monday, 6=Sunday
<code>date_received_quarter</code>	1..4

- **HINTS:**
 - Use the `.dt` accessor on a datetime column.
 - `df['date_received'].dt.year, .dt.month, .dt.dayofweek, .dt.quarter`.
- **WHY:** A model can learn “mortgage complaints spike in 2008” only if YOU give it the `date_received_year` column.

Step 3 — Confirm `days_to_company_response`

- **WHAT:** You made this in Class 2 Step 6. Check it is still there. If not, recompute.
- **HINTS:** `(df['date_sent_to_company'] - df['date_received']).dt.days.clip(0, 365)`.

Step 4 — Confirm narrative features

- **WHAT:** These come from Class 2. Check they exist:
 - `narrative_length`
 - `narrative_n_caps_words`
 - `narrative_intensity_score`
 - `n_dollar_mentions`
 - `n_dates_mentioned`
 - `has_narrative_flag`
- If any is missing, recompute.

Step 5 — Re-verify company_target_encoded

- **WHAT:** From Class 2. Re-check no missing values.
- **HINTS:** `df['company_target_encoded'].isna().sum()` should be 0.
- **If missing exist:** fill with the overall train mean.

Step 6 — Engineer is_recent

- **WHAT:** A binary column: 1 if `date_received_year >= 2020`, else 0.
- **WHY:** Complaint patterns changed after COVID. The model should be able to use this as a fast switch.
- **HINTS:** `df['is_recent'] = (df['date_received_year'] >= 2020).astype(int)`.

Step 7 — Engineer narrative_word_count

- **WHAT:** Count words in the narrative (split on whitespace).
- **HINTS:**
 - `df['narrative_word_count'] = df['narrative'].fillna('').str.split().str.len()`.
- **WHY:** Character length and word count tell different stories. (One huge word vs many short words.)

Step 8 — Engineer narrative_has_xxxx

- **WHAT:** Binary: 1 if narrative contains XXXX, else 0.
- **HINTS:**
 - `df['narrative_has_xxxx'] = df['narrative'].fillna('').str.contains('XXXX').astype(int)`.
- **WHY:** A narrative with redactions probably mentions personal info (account number, name). Mortgages and bank complaints mention accounts; credit-reporting complaints mention SSN. Useful signal.

Step 9 — Save

- **HINTS:** `df.to_parquet('/content/drive/MyDrive/cfpb_lab/complaints_step3.parquet')`.

Phase C — Make ONE chart for Diana (15 minutes)

Diana's chart — “Product mix by US state”

A stacked bar chart showing the top 10 US states and the product mix in each.

- **HINTS:**
 - First, get top 10 states by total volume:
`top_states = df['state'].value_counts().head(10).index`
`sub = df[df['state'].isin(top_states)]`
 - Then: `pd.crosstab(sub['state'], sub['product'], normalize='index')`.
 - Plot as `.plot.bar(stacked=True)`.
- **Title:** “Product mix in the 10 most complaining states”.
- **X-label:** “State”.
- **Y-label:** “Share of complaints (0 to 1)”.
- **Takeaway for Diana:** “California complaints are mostly about credit reporting. Florida is mostly mortgage. Build state-specific teams.”

Common mistakes in Class 3

Mistake	What goes wrong
Forget <code>.dt.days</code> and get a <code>Timedelta</code>	Model crashes on <code>Timedelta</code> dtype.

Mistake	What goes wrong
Use <code>.dt.dayofweek</code> on a string column	<code>AttributeError</code> . The column must be datetime first.
Count XXXX as caps words (again)	This was the Class 2 mistake. Re-check.
Compute company target-encoding on full data	Leakage. Train-only.
Drop the narrative column entirely	Module 7 cannot run without it.

Self-check before Class 4

- ☐ 4 date-derived features exist.
- ☐ `days_to_company_response` exists, no missing.
- ☐ All 5 narrative numeric features exist.
- ☐ `company_target_encoded` exists, no missing.
- ☐ `is_recent`, `narrative_word_count`, `narrative_has_xxxx` exist.
- ☐ Narrative text column is STILL there (for Module 7).
- ☐ 3+ exploratory charts.
- ☐ 1 explanatory chart.
- ☐ `complaints_step3.parquet` saved to Drive.

Class 4 — Feature Selection

Scenario reminder: Now you have ~22 columns. Diana says: “Too many. Some are duplicates. Some are useless. I want 15 GOOD columns plus the narrative. Pick them.”

Your goal

Pick the best ~15 columns. Drop the rest. Justify every choice.

Inputs

- `complaints_step3.parquet`

Outputs

- `complaints_step4.parquet` in Drive (only the selected columns + product + narrative)
- 3+ exploratory charts (correlation heatmap, mutual info bars, importance bars)
- A short markdown report explaining your choices

Phase A — Explore the data first (15 minutes)

Exploratory chart 1 — Correlation heatmap of numeric columns

- **Question:** “Which columns say the same thing as each other?”
- **HINTS:**
 - Pick all numeric columns: `numeric_cols = df.select_dtypes(include='number').columns`.
 - `df[numeric_cols].corr()`.
 - `sns.heatmap()` with `annot=True` and `fmt='.2f'`.
- **What you learn:** `narrative_length` and `narrative_word_count` should be highly correlated (~0.95). Pick ONE.

Exploratory chart 2 — Mutual information bar chart

- After you compute mutual info (Step 4 below), plot it.
- **HINTS:**
 - Sort the values, then bar chart.
- **What you learn:** Which columns predict product strongest. Expect company_target_encoded and narrative features at the top.

Exploratory chart 3 — Random Forest feature importance

- After you train the RF (Step 5 below), plot the importances.
 - **HINTS:** `pd.Series(rf.feature_importances_, index=...).sort_values().plot.barh()`.
 - **What you learn:** A second opinion on feature importance.
-

Phase B — Select features (50 minutes)

Step 1 — Re-create the train/test split

- **WHAT:** You did this in Class 2, but the split was on a different DataFrame. Redo on `complaints_step3.parquet`. Use the SAME `random_state=42` for reproducibility.
- **HINTS:**
 - `from sklearn.model_selection import train_test_split.`
 - `X = df.drop(['product', 'narrative'], axis=1).` (Drop the target AND the raw text — text is for M7.)
 - `y = df['product'].`
 - Encode y again: `LabelEncoder().fit_transform(y).`
 - `train_test_split(X, y, test_size=0.2, random_state=42, stratify=y).`

Step 2 — Remove low-variance columns

- **WHAT:** Drop columns where almost all values are the same. Example: a column that is 99% zero.
- **HINTS:**
 - `from sklearn.feature_selection import VarianceThreshold.`
 - `VarianceThreshold(threshold=0.01).fit(X_train[numeric_cols]).`
 - `.get_support()` returns True/False per column.
- **EXPECTED:** Maybe 0-2 columns dropped.

Step 3 — Remove highly correlated columns

- **WHAT:** Drop one of each pair where $|\text{corr}| > 0.9$.
- **HINTS:**
 - Compute correlation matrix.
 - Get upper triangle with `np.triu(...).`
 - For each column, if any of its upper-triangle correlations is > 0.9 , mark it for dropping.
- **EXPECTED:** You probably drop `narrative_word_count` (it duplicates `narrative_length`).

Step 4 — Rank by mutual information

- **WHAT:** Score each column by how much it tells you about product.
- **HINTS:**
 - `from sklearn.feature_selection import mutual_info_classif.`
 - `mi = mutual_info_classif(X_train_numeric, y_train_enc).`
 - Put in a Series, sort.
- **EXPECTED:** `company_target_encoded`, `narrative_length`, `n_dollar_mentions`, `date_received_year` should be at the top.

Step 5 — Random Forest importance (second opinion)

- **WHAT:** Train a small Random Forest, look at `feature_importances_`.
- **HINTS:**
 - `from sklearn.ensemble import RandomForestClassifier.`
 - `RandomForestClassifier(n_estimators=100, max_depth=10, n_jobs=-1, class_weight='balanced', random_state=42).`
 - `.fit(X_train_numeric, y_train_enc).`
 - Look at `.feature_importances_`.
- **WARNING:** This may take 2-3 minutes on 160,000 rows. Use `n_jobs=-1` to use all CPU cores.

Step 6 — Pick the final 15 columns

- **WHAT:** Combine the rankings. Pick columns that:
 - Are high in mutual info, AND
 - Are high in RF importance, AND
 - Were not dropped by variance/correlation pruning.
- **WRITE DOWN:** In your notebook, list the columns. Explain in 1 sentence why each one is in.

A reasonable final list (you may pick differently):

#	Column	Why
1	complaint_id	ID (not for model, for tracking)
2	date_received_year	Era of complaint
3	date_received_month	Seasonality
4	date_received_dayofweek	Weekend effect
5	days_to_company_response	Backlog signal
6	product	TARGET
7	sub_product	Sub-category hint
8	issue	Strong text-y label
9	company_response_type	One-hot
10	state	Geography
11	has_narrative_flag	Did citizen write text?
12	narrative_length	Length of text
13	narrative_n_caps_words	Anger signal
14	narrative_intensity_score	Anger ratio
15	n_dollar_mentions	Money signal
16	n_dates_mentioned	Specificity signal
17	company_target_encoded	Company identity, compressed
18	consumer_disputed	1/0/-1
19	timely_response	1/0
20	is_recent	Post-COVID flag
21	narrative_has_xxxx	Redaction flag
22	narrative	Raw text - KEPT for Module 7

That is 22 columns. Diana wanted ~20-22. Good.

Step 7 — Save

- **HINTS:** Keep only the selected columns. Save as `complaints_step4.parquet` to Drive.

Phase C — Make ONE chart for Diana (15 minutes)

Diana's chart — “These are the 10 most important columns for routing”

A horizontal bar chart of your top 10 features and their importance score.

- **HINTS:**
 - Use the RF importance.
 - `sort_values()` then `.tail(10)` then `.plot.barh()`.
 - **Title:** “Top 10 predictors of complaint product category.”
 - **Y-axis:** column names.
 - **X-axis:** Random Forest importance.
 - **Takeaway for Diana:** “The company name (target-encoded) and the narrative length predict 60% of routing. With just these two signals we already beat random guessing 6 times.”
-

Common mistakes in Class 4

Mistake	What goes wrong
Compute mutual info on the FULL data	Leakage.
Drop columns without writing why	Module 4 students will not understand.
Keep too many columns (40+)	Slow training. Overfitting risk.
Drop the raw narrative column	Module 7 cannot run without it.
Drop <code>complaint_id</code>	OK to drop from the FEATURES but KEEP it in the file (for tracking).

Self-check before Class 5

- ☐ Train/test split done FIRST.
 - ☐ ~15 numeric/categorical columns selected + product + narrative + `complaint_id`.
 - ☐ Total ~20-22 columns in the file.
 - ☐ You wrote down WHY for each kept column.
 - ☐ 3+ exploratory charts.
 - ☐ 1 chart for Diana.
 - ☐ `complaints_step4.parquet` saved to Drive.
-

Class 5 — Pipelines

Scenario reminder: Diana says: “Your cleaning code is in 4 different notebooks. When a new complaint arrives tomorrow at 9am, you cannot copy 4 notebooks to the server. We need ONE object that does everything.”

Your goal

Put EVERY cleaning step inside ONE Pipeline object. Production-ready code.

Inputs

- `complaints_step4.parquet` (selected columns)

Outputs

- `cfpb_pipeline.joblib` saved in Drive
 - 1 confusion-matrix chart (10x10) + 1 per-class F1 chart
 - A pipeline that takes RAW input and produces a predicted product
-

Phase A — Explore the data first (15 minutes)

Exploratory chart 1 — Confusion matrix (after training)

- After Step 6, build a confusion matrix.
- **HINTS:**
 - `from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay.`
 - `cm = confusion_matrix(y_test, y_pred).`
 - `ConfusionMatrixDisplay(cm, display_labels=le.classes_).plot(xticks_rotation=45).`
- **What you learn:** Which products does the model confuse? Mortgage vs Mortgage broker? Credit reporting vs Credit card?

Exploratory chart 2 — Per-class F1 score bar chart

- **HINTS:**
 - `from sklearn.metrics import classification_report.`
 - `report = classification_report(y_test, y_pred, output_dict=True).`
 - Pull out the F1 per class. Bar chart.
 - **What you learn:** Big classes (Mortgage) have high F1. Small classes (Money transfer) have low F1. This is the **imbalance problem**.
-

Phase B — Build the pipeline (50 minutes)

Step 1 — Decide which columns are numeric and which are categorical

- **HINTS:** Make two lists. Drop `complaint_id` and `narrative` from the feature lists.
- **EXAMPLE:**
 - numeric: `date_received_year, date_received_month, date_received_dayofweek, days_to_company_response, narrative_length, narrative_n_caps_words, narrative_intensity_score, n_dollar_mentions, n_dates_mentioned, company_target_encoded, has_narrative_flag, consumer_disputed, timely_response, is_recent, narrative_has_xxxx.`
 - categorical: `sub_product, issue, company_response_type, state.`

Step 2 — Build the numeric mini-pipeline

- **WHAT:** 2 steps: imputer (fill missing) + scaler.
- **HINTS:**
 - `from sklearn.pipeline import Pipeline.`
 - `from sklearn.impute import SimpleImputer.`
 - `from sklearn.preprocessing import StandardScaler.`
 - Skeleton:

```
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='___')),
    ('scaler', StandardScaler()),
])
```
 - Fill in the blank: best strategy for numeric? ('median')

Step 3 — Build the categorical mini-pipeline

- **WHAT:** 2 steps: imputer + one-hot encoder.
- **HINTS:**
 - from sklearn.preprocessing import OneHotEncoder.
 - Skeleton:

```
categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='__', fill_value='unknown')),
    ('onehot', OneHotEncoder(handle_unknown='__', sparse_output=__)),
])
```
 - Fill in blanks. (Strategy 'constant', handle_unknown 'ignore', sparse_output False.)
- **WHY handle_unknown='ignore'?** In production, a new company response type or US state may appear. We do not want the code to crash.

Step 4 — Combine into a ColumnTransformer

- **HINTS:**
 - from sklearn.compose import ColumnTransformer.
 - It takes a list of tuples: (name, transformer, list_of_columns).
 - Skeleton:

```
preprocessor = ColumnTransformer(transformers=[
    ('num', numeric_transformer, numeric_cols),
    ('cat', categorical_transformer, categorical_cols),
])
```

Step 5 — Add the model on top

- **HINTS:**
 - from sklearn.linear_model import LogisticRegression.
 - LogisticRegression(class_weight='balanced', max_iter=1000, multi_class='multinomial', solver='lbfgs', random_state=42, n_jobs=-1).
 - Put it in a Pipeline with the preprocessor.
- **WHY class_weight='balanced'?** Mortgage is 30% of data. Money transfer is 1%. Without balancing, the model predicts "Mortgage" for almost everything.
- **WHY multi_class='multinomial'?** Because we have 10 classes, not 2.

Step 6 — Train and evaluate

- **HINTS:**
 - full_pipeline.fit(X_train, y_train_enc).
 - y_pred = full_pipeline.predict(X_test).
 - from sklearn.metrics import classification_report.
 - print(classification_report(y_test_enc, y_pred, target_names=le.classes_)).
- **EXPECTED:** Overall accuracy around 60-70%. Mortgage F1 ~0.85. Smallest class F1 ~0.20. (Module 4 improves this with a stronger model and SMOTE.)

Step 7 — Save the trained pipeline

- **HINTS:**
 - import joblib.
 - joblib.dump(full_pipeline, '/content/drive/MyDrive/cfpb_lab/cfpb_pipeline.joblib').
 - Also save the label encoder: joblib.dump(le, '/content/drive/MyDrive/cfpb_lab/cfpb_label_encoder.joblib').

Phase C — Make ONE chart for Diana (15 minutes)

Diana's chart — “How well do we route each product?”

A horizontal bar chart of per-class F1 scores.

- **HINTS:**
 - From the classification report, pull each class's F1.
 - Sort. Bar chart.
 - **Title:** “Routing accuracy by product (baseline model)”.
 - **X-axis:** F1 score (0 to 1).
 - **Y-axis:** product name.
 - **Takeaway for Diana:** “We route Mortgage and Credit reporting at 85% F1. We are weak on Money transfer (20%) because we only have 2,000 examples. Module 4 with SMOTE will help.”
-

Common mistakes in Class 5

Mistake	What goes wrong
Build the pipeline AFTER you scaled the data manually	Pipeline scales it twice. Numbers wrong.
Forget <code>sparse_output=False</code> in <code>OneHotEncoder</code>	Some downstream code breaks.
Use <code>class_weight='balanced'</code> AND SMOTE	Over-correction. Pick one. Save SMOTE for Module 4.
Forget <code>random_state</code>	Results change every run. Hard to debug.
Forget to save the <code>LabelEncoder</code>	You can predict numbers but not product names.
Train on the narrative text column	Logistic Regression on raw text crashes. Drop it before fit.

Self-check before Class 6

- ☐ Pipeline has preprocessor + model.
 - ☐ Preprocessor has numeric + categorical transformers.
 - ☐ `handle_unknown='ignore'` set.
 - ☐ `class_weight='balanced'` set.
 - ☐ `multi_class='multinomial'` set.
 - ☐ Confusion matrix (10x10) + per-class F1 chart in your notebook.
 - ☐ `cfpb_pipeline.joblib` saved to Drive.
 - ☐ `cfpb_label_encoder.joblib` saved to Drive.
-

Class 6 — End-to-End Lab (THE BIG ONE)

Scenario reminder: Diana is in the meeting room. She wants the FINAL clean dataset on her desk in 100 minutes. This is the lab.

Your goal

Take the raw CFPB complaints CSV. Produce ONE final .parquet file with the exact 22-column schema. Plus a 1-page findings report.

Time

100 minutes of focused work. (10 more than usual because of the size of the data and the 10-class target.)

Inputs

- `complaints_sample_200k.csv` in your Drive folder

Outputs

- `cfpb_complaints_clean.parquet` (~200,000 rows x 22 columns) in Drive
 - `findings.md` (~1 page)
 - Your full Colab notebook
-

Phase A — Explore the data first (15 minutes)

You will reuse charts from Classes 1-5. Pick 3 of them. Put them all on ONE PAGE of your notebook with markdown headers:

1. Complaint volume over time (Class 1) - the backlog story.
2. Top 10 complained-about companies (Class 2) - who CFPB is watching.
3. Product mix per US state (Class 3) - the routing map.

These 3 charts tell Diana the whole story.

Phase B — Build the final dataset (75 minutes)

Required output schema

Your `cfpb_complaints_clean.parquet` MUST have these 22 columns, with these names and types:

#	Column	Type	Source
1	<code>complaint_id</code>	int / string	raw CFPB
2	<code>date_received_year</code>	int	engineered
3	<code>date_received_month</code>	int	engineered
4	<code>date_received_dayofweek</code>	int	engineered
5	<code>days_to_company_response</code>	int (0-365)	engineered
6	<code>product</code>	string (10 categories)	TARGET (cleaned)
7	<code>sub_product</code>	string	raw
8	<code>issue</code>	string	raw
9	<code>company_response_type</code>	string	raw
10	<code>state</code>	string (52 incl. territories)	raw
11	<code>has_narrative_flag</code>	int (0/1)	engineered
12	<code>narrative_length</code>	int	engineered
13	<code>narrative_n_caps_words</code>	int	engineered
14	<code>narrative_intensity_score</code>	float	engineered
15	<code>n_dollar_mentions</code>	int	engineered
16	<code>n_dates_mentioned</code>	int	engineered
17	<code>company_target_encoded</code>	float	engineered (TRAIN ONLY mean)
18	<code>consumer_disputed</code>	int (-1/0/1)	engineered
19	<code>timely_response</code>	int (0/1)	raw -> binary
20	<code>is_recent</code>	int (0/1)	engineered
21	<code>narrative_has_xxxx</code>	int (0/1)	engineered
22	<code>narrative</code>	string	raw (kept for Module 7)

Lab phases (timed)

Phase	Time	What you do
1. Load + rename	10 min	Load sample CSV. Rename to snake_case. Drop tags, zip_code.
2. Clean dates	10 min	Convert 2 date columns. Add date_received_year/month/dayofweek. Compute
3. Clean product/state	10 min	days_to_company_response. Merge legacy product names into 10 categories. Fill state missing with "XX".
4. Handle narrative + missing	15 min	Fill empty narratives with "". Make has_narrative_flag. Make narrative_has_xxxx.
5. Numeric narrative features	15 min	Compute narrative_length, narrative_n_caps_words, narrative_intensity_score, n_dollar_mentions, n_dates_mentioned.
6. Encode + target-encode	10 min	Binary timely_response, consumer_disputed. Target-encode company (TRAIN MEAN ONLY).
7. Save + validate	5 min	Drop unused columns. Confirm 22 columns. Save .parquet.
8. Findings	10 min	Write findings.md.

Total: 85 minutes work + 15 minutes for charts = 100 minutes.

Phase C — Findings report for Diana (10 minutes)

Write findings.md with these sections:

Final numbers

- Final row count: ____ (should be ~200,000)
- Number of unique products: ____ (should be 10)
- % of rows with non-empty narrative: ____% (should be ~35%)
- Largest product class % share: ____% (Mortgage or Credit reporting)
- Smallest product class % share: ____% (Money transfer or Prepaid card)

Top 3 insights

1. _____
2. _____
3. _____

One question to investigate in Module 4

- _____

One chart that summarizes everything

Embed your most important chart (the product mix per state stacked bar).

Grading rubric (100 points)

Item	Points
All 22 columns exist with the right names and dtypes	20
Cleaning decisions written in markdown	10
Pipeline is reproducible (re-run from raw CSV in ONE command)	15
product has exactly 10 categories after cleaning	10
company_target_encoded was computed on TRAIN ONLY (no leakage)	10
Narrative kept (NOT dropped) and has_narrative_flag exists	10
At least 3 exploratory + 1 explanatory chart per class	15
findings.md has insights, not just numbers	10

Submit

Upload to module-3/class_6/submissions/<YourTeamName>/:

- cfpb_complaints_clean.parquet
 - Your Colab notebook (File --> Download --> Download .ipynb)
 - findings.md
-

Tips for the whole module

1. **Do NOT copy-paste code from this guide.** This guide gives you HINTS only. Write the code yourself. You will learn much faster.
2. **Sample early, save early.** The full 4M-row CSV will crash free Colab. Sample to 200k once, save to Drive, reuse.
3. **Save your notebook to Drive often.** Colab disconnects after 12 hours.
4. **Save intermediate .parquet files to Drive** (complaints_step1, complaints_step2, etc.). If something breaks, you do not redo everything.
5. **Never target-encode on the full data.** Always train-only. This single mistake makes the model look great in class and fail in production. Write the rule on a sticky note on your monitor.
6. **Pair-program.** One student types, one reads. Switch every 20 minutes.
7. **Make a chart BEFORE you write cleaning code.** You must SEE the problem before you can fix it.
8. **Make a chart AFTER each step.** Confirm your code did what you expected.
9. **Watch for XXXX.** Almost every narrative feature you compute is affected by CFPB's redaction tokens. Always replace XXXX with a space BEFORE measuring caps words or word counts.
10. **Ask the mentor early.** If you are stuck for 20 minutes, ask. Do not waste an afternoon.

This is the hardest scenario in the curriculum. If you finish this one, every future lab will feel easy.

Good luck.